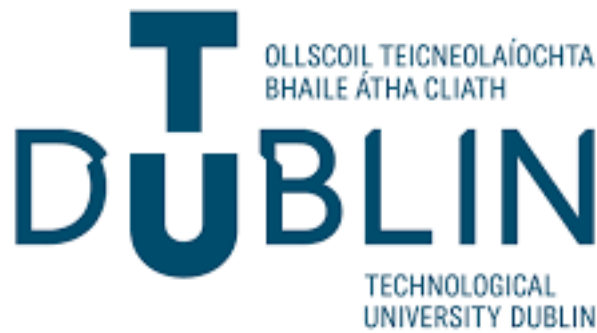




# **Systems Software Report Supplemental CA1**

**TU856  
BSc in Computer Science**

**Jared Leonard  
C21468162**

School of Computer Science  
TU Dublin – City Campus

**20 March 2025**

## Table of Contents

|                                                                     |   |
|---------------------------------------------------------------------|---|
| <i>Functionality Checklist</i> .....                                | 3 |
| <i>Feature 1 - System Architecture including makefile</i> .....     | 4 |
| <i>Feature 2 - Daemon (Setup/ Initialisation/ Management)</i> ..... | 4 |
| <i>Feature 3 - Daemon (Implementation)</i> .....                    | 4 |
| <i>Feature 4 - Backup Functionality</i> .....                       | 5 |
| <i>Feature 5 - Transfer Functionality</i> .....                     | 5 |
| <i>Feature 6 - Lockdown directories for Backup / Transfer</i> ..... | 5 |
| <i>Feature 7 – Process management and IPC</i> .....                 | 5 |
| <i>Feature 8 - Logging and Error Logging</i> .....                  | 5 |
| <i>Conclusion</i> .....                                             | 6 |

## Functionality Checklist

| <b>Feature</b> | <b>Description</b>                       | <b>Implemented</b> |
|----------------|------------------------------------------|--------------------|
| F1             | System Architecture including makefile   | Yes or No          |
| F2             | Daemon (Setup/Initialisation/Management) | Yes or No          |
| F3             | Daemon (Implementation)                  | Yes or No          |
| F4             | Backup Functionality                     | Yes or No          |
| F5             | Transfer Functionality                   | Yes or No          |
| F6             | Lockdown folder for Backup / Transfer    | Yes or No          |
| F7             | Process management and IPC               | Yes or No          |
| F8             | Logging and Error Logging                | Yes or No          |

Have you included a video demo as part of the assignment: Yes or No

Link to Video: <https://youtu.be/UOMg0o5XI4M>

---

## Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

Jared Leonard

20 March 2025

## *Feature 1 - System Architecture including makefile*

My system was designed with a modular architecture, following the SoC and SRP, where each component has a specific responsibility.

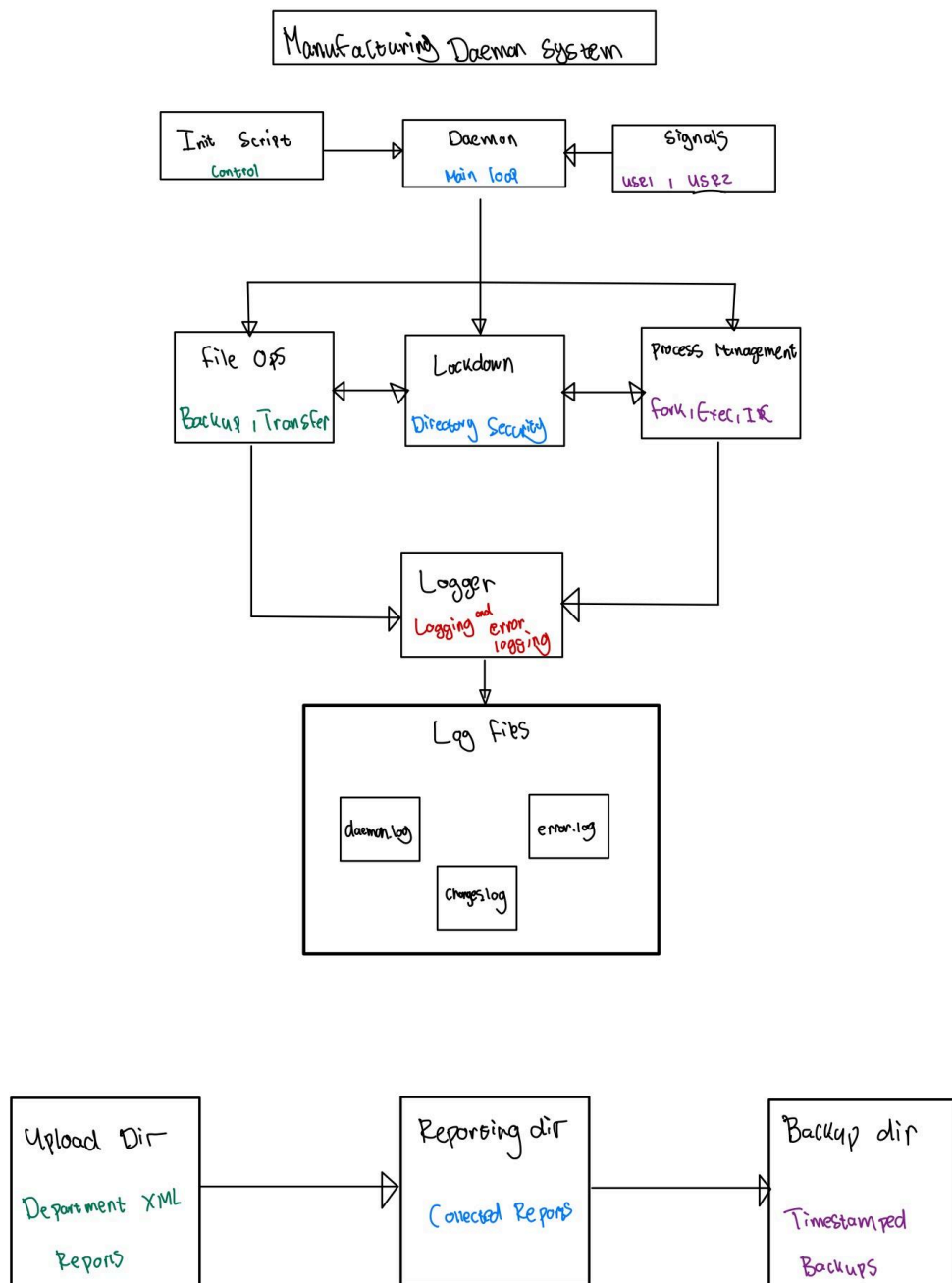
- **daemon.c** - Main daemon process and control flow file
- **fileops.c** - Responsible for file operations, including backup and transfer functionality.
- **lockdown.c** - Provides the security mechanism for directory locking
- **logger.c** - Comprehensive logging systems are created in this file
- **ipc.c** - In charge of process management and inter-process communication
- **config.h** A centralized header file for our config variables

By using this modular design, I ensured that each component is responsible for handling a specific aspect of the system, which made the code easier to maintain and debug. The architecture of the system provided clear separation between the daemon's core functions and the specific operations it provides.

I created a makefile to automate the build process, and to ensure all the dependencies are properly managed. This included the following:

- The compilation of all source files
- Dependency tracking for efficient builds
- Linking of object files

### **Architecture Diagram**



## Feature 2 - Daemon (Setup/ Initialisation/ Management)

I ensured that the daemon setup followed best practices for Linux daemon implementation.

### **Startup script and init process:**

I created the script following the SysV init script standard, where the script supports start, stop, restart, status, backup, and transfer operations. For each operation, I returned proper exit codes. I implemented the script using the singleton pattern by checking for an already existing PID file, to ensure there is only a single daemon running at a time. The configuration of the system is handled via the config.h header file.

### **Daemon Control Options:**

- start: initializes the daemon if it is not running
- stop: terminates the daemon gracefully
- restart: stops and restarts the daemon
- status: reports the current state of the daemon
- backup: triggers an instant backup operation
- transfer: triggers an instant transfer operation

The daemon creates a PID file, which ensures that there is only one instance of the daemon running at any given time. This is done so via the singleton pattern. By doing this, I ensure that there is stable and reliable operations and no resource conflicts.

### *Feature 3 - Daemon (Implementation)*

The implementation of the daemon follows the standard UNIX process creation pattern where we firstly fork the parent process, and create a new session with `setsid()`. Following this, we fork again to ensure that the daemon cannot reacquire a controlling terminal. The working directory is changed to the root directory and all open file descriptors are closed. Standard I/O is redirected to `/dev/null`. Finally, signal handlers are initialized.

The daemon is decoupled from the terminal as well as the I/O, running as a background process. Another key aspect of the implementation is signal handling:

- SIGTERM: Graceful shutdown
- SIGUSR1: Trigger backup operation
- SIGUSR2: Trigger transfer operation
- SIGCHLD: Handle child process completion

I implemented error handling via appropriate logging. The daemon checks for and handles error conditions that include: file system errors, permission issues, process creation failures, and signal handling failures.

### *Feature 4 - Backup Functionality*

So the backup system creates reliable backups of the reporting directory.

### **Implementation:**

- all the backups are created in timestamped directories
- all files are error checked before being copied

- the original attributes of the file are kept
- the backup directories are created with the appropriate permissions.

Error handling was incorporated as well as part of the functionality, with the most important being permission access and the failure of directory creations.

#### **Automation**

- backups can be triggered manually via SIGUSR1 signal handler.
- All backup operations are logged
- The locking of directories ensures data consistency during the backup operation

### *Feature 5 - Transfer Functionality*

The transfer system is in charge of moving XML files from the upload directory to the reporting directory.

#### **Implementation:**

- files are moved via atomic operations, where the operation cannot be interrupted, where possible.
- During the transfer of the files, the file metadata is also preserved
- the user info of each transferred file is logged
- for auditing purposes, timestamps are recorded.

I incorporated error handling via file not found errors, permission issues, destination conflicts, and cross-filesystem transfers.

#### **Automation**

- transfers can be triggered manually via the SIGUSR2 signal handler.
- Missing uploads for departments are logged
- All transfer operations are logged

The transfer operation verifies file integrity before and after operations are completed to make sure that all the department reports are processed correctly. Important info is recorded on the system for audit purposes (user, timestamps etc)

### *Feature 6 - Lockdown directories for Backup / Transfer*

This mechanism ensures data consistency and security during operations.

#### **Implementation:**

- to prevent the race condition, operations are atomic
- after operations are complete, permissions are restored.
- directories are locked by changing the permission to read only
- for permission changes, we incorporate proper error handling

We have to keep some security considerations in mind, such as ensuring that only the daemon process can modify the directories during operations and that user cannot interfere with operations that are taking place. The system also should maintain a consistent state even if it is interrupted.

The lockdown mechanism protects the system during critical operations, making sure that the backup and transfer operations are reliable, and that there is no data corruption.

### *Feature 7 – Process management and IPC*

Process management is handled using the standard UNIX process creation and handling mechanisms.

#### **Implementation:**

- Child processes are created using `fork()`
- `exec()` functions are used to replace process images
- `waitpid()` handling is used for child process completion
- signal based communication between processes takes place

The IPC mechanism communicates success or failure via exit codes. The parent process checks the completion status of the child process. We use signals to trigger the operations that take place. Finally, zombie processes are handled.

Error handling is incorporated via execution failures, process creation failures, and if there are any abnormal terminations of processes.

This system makes sure that each operation will run within an independent process space, which improves reliability and prevents failure of one operation from negatively affecting others

### *Feature 8 - Logging and Error Logging*

The logging system is used to track all of the daemon operations.

#### **Implementation:**

- Multiple log files are created, including the `daemon.log` file for general operation logs, the `error.log` file for error specific logging, and the `changes.log` file for file modification tracking.
- Log levels include `DEBUG`, `INFO`, `WARNING`, `ERROR`
- There are timestamped entries within each logging operation, along with user id.

For error logging in particular, there is detailed error messages with error code, with more info about what the error may be to allow us to debug easier. In my video demo.

The change logging system shows us the username of the user modifying the system state, as well as file details, the timestamp of when the modification took place, and what the operation type was.

This system provides us with transparency about the daemon's operations, which allows us to monitor, debug, and audit our system effectively. All system events, ranging from normal

operations to critical errors are captured with comprehensive detail to help us understand the system behaviour well

## *Conclusion*

The manufacturing daemon I developed addresses all the requirements set out by the CTO, including:

1. A daemon process for managing department reports
2. The secure transfer of files from the upload to reporting directories
3. Automated backup of reporting files
4. Logging of operations and changes made
5. The prevention of modifications during critical operations.

I overcame challenges to ensure the following:

- the daemon can run reliably in the background
- secure directory locking mechanisms
- efficient and effective handling of processes management and IPC
- the creation of a logging system

This implementation ensures that the company's reporting system has a solid foundation, ensuring that all file operations are reliable, secure, and transparent